Name: I Raiyan

Reg no: 192521421

CSA0918

1. Most HackerRank challenges require you to read input from stdin (standard input) and write output to stdout (standard output).

One popular way to read input from stdin is by using the Scanner class and specifying the *Input Stream* as *System.in*. For example:

```
Scanner scanner = new Scanner(System.in);

String myString = scanner.next();

int myInt = scanner.nextInt();

scanner.close();


System.out.println("myString is: " + myString);

System.out.println("myInt is: " + myInt);
```

The code above creates a *Scanner* object named and uses it to read a *String* and an *int*. It then *closes* the *Scanner* object because there is no more input to read, and prints to stdout using *System.out.println(String)*. So, if our input is:

```
Hi 5
```

Our code will print:

```
myString is: Hi

myInt is: 5
```

Alternatively, you can use the BufferedReader class.

Task

In this challenge, you must read integers from stdin and then print them to

stdout. Each integer must be printed on a new line. To make the problem a little easier, a portion of the code is provided for you in the editor below.

Input Format

There are lines of input, and each line contains a single integer.

Sample Input

42

100

125

Sample Output

42

100

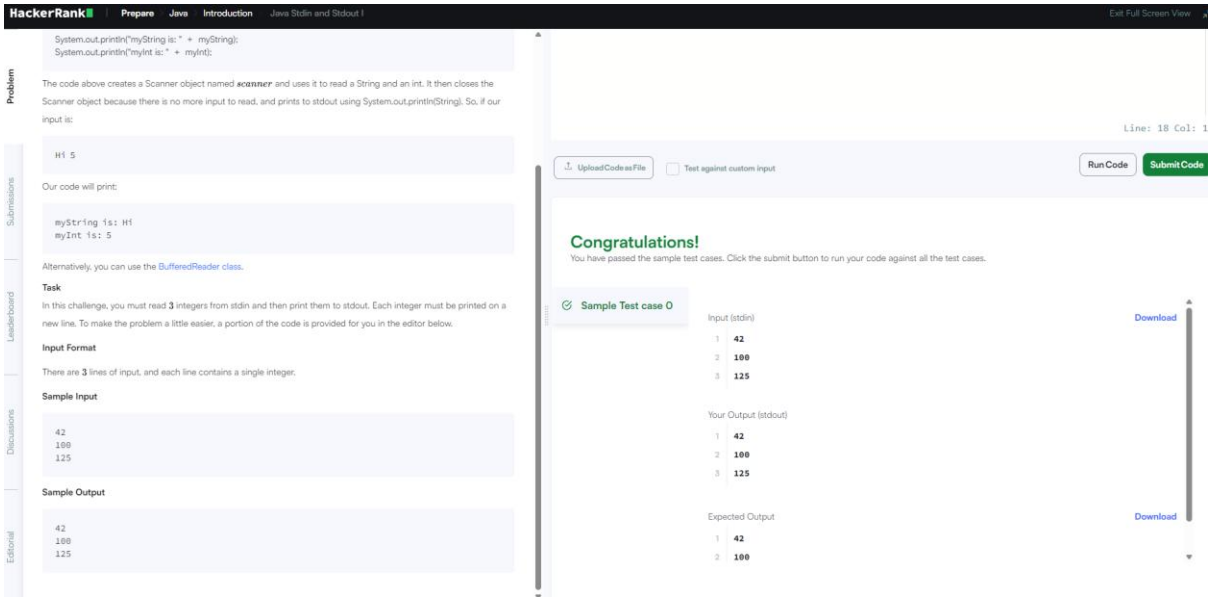
125

Code:

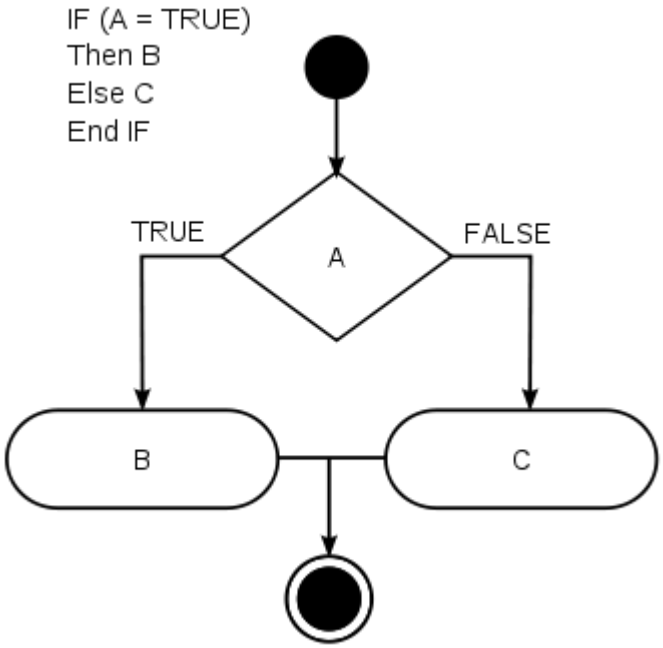
```
import java.util.Scanner;


public class Solution {
 public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


      int a = sc.nextInt();
       int b = sc.nextInt();
        int c = sc.nextInt();


         System.out.println(a);
         System.out.println(b);
```

```
System.out.println(c);


sc.close();
    }
        }
```

Output:



2. In this challenge, we test your knowledge of using *if-else* conditional statements to automate decision-making processes. An if-else statement has the following logical flow:



Task

Given an integer, , perform the following conditional actions:

- If is odd, print Weird
- If is even and in the inclusive range of to , print Not Weird
- If is even and in the inclusive range of to , print Weird
- If is even and greater than , print Not Weird

Complete the stub code provided in your editor to print whether or not is weird.

Input Format

A single line containing a positive integer, .

Constraints

-

Output Format

Print Weird if the number is weird; otherwise, print Not Weird.

Sample Input 0

3

Sample Output 0

Weird

Sample Input 1

24

Sample Output 1

Not Weird

Explanation

Sample Case 0:
 is odd and odd numbers are weird, so we print Weird.

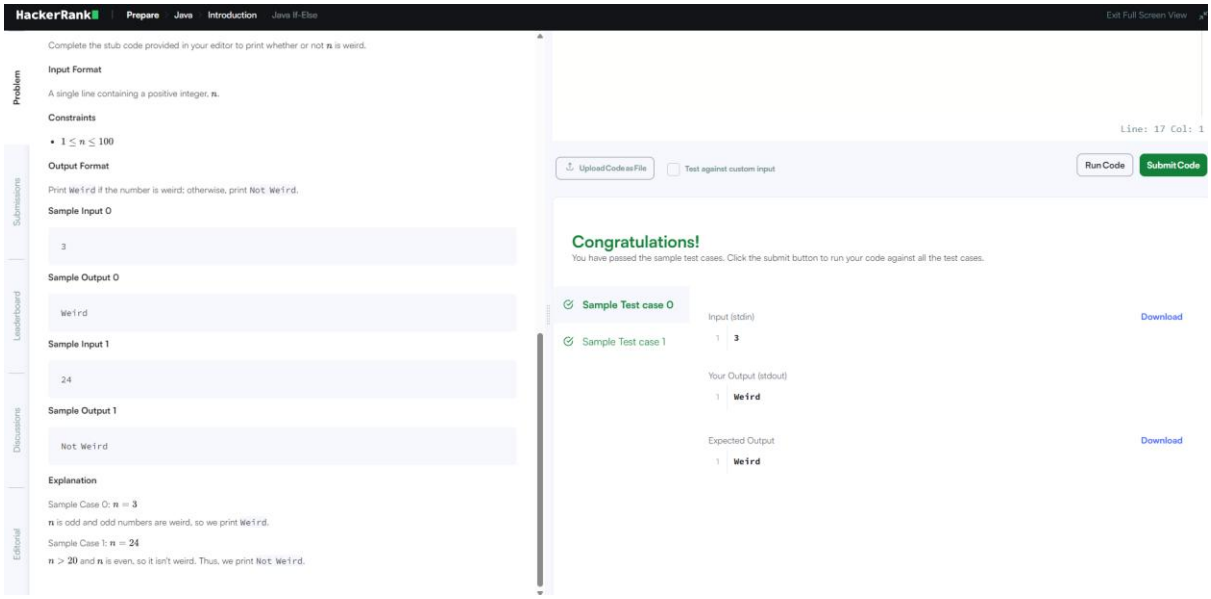
Sample Case 1:
 and is even, so it isn't weird. Thus, we print Not Weird.

CODE:

```
import java.util.Scanner;

public class Solution{

public static void main(String[] args){

Scanner sc=new Scanner(System.in);

int n=sc.nextInt();

if(n%2!=0)
```

```
System.out.println("Weird");
else if(n>=2&&n<=5)
System.out.println("Not Weird");
else if(n>=6&&n<=20)
System.out.println("Weird");
else
System.out.println("Not Weird");
sc.close();
}
}
```

OUTPUT:



3. In this challenge, you must read an *integer*, a *double*, and a *String* from stdin, then print the values according to the instructions in the *Output Format* section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing Java Stdin and Stdout I before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an *integer*.
2. The second line contains a *double*.
3. The third line contains a *String*.

Output Format

There are three lines of output:

1. On the first line, print String: followed by the unaltered *String* read from stdin.
2. On the second line, print Double: followed by the unaltered *double* read from stdin.
3. On the third line, print Int: followed by the unaltered *integer* read from stdin.

To make the problem easier, a portion of the code is already provided in the editor.

Note: If you use the *nextLine()* method immediately following the *nextInt()* method, recall that *nextInt()* reads integer tokens; because of this, the last newline character for that line of integer input is still queued in the input buffer and the next *nextLine()* will be reading the remainder of the integer line (which is empty).

Sample Input

42

3.1415

Welcome to HackerRank's Java tutorials!

Sample Output

String: Welcome to HackerRank's Java tutorials!

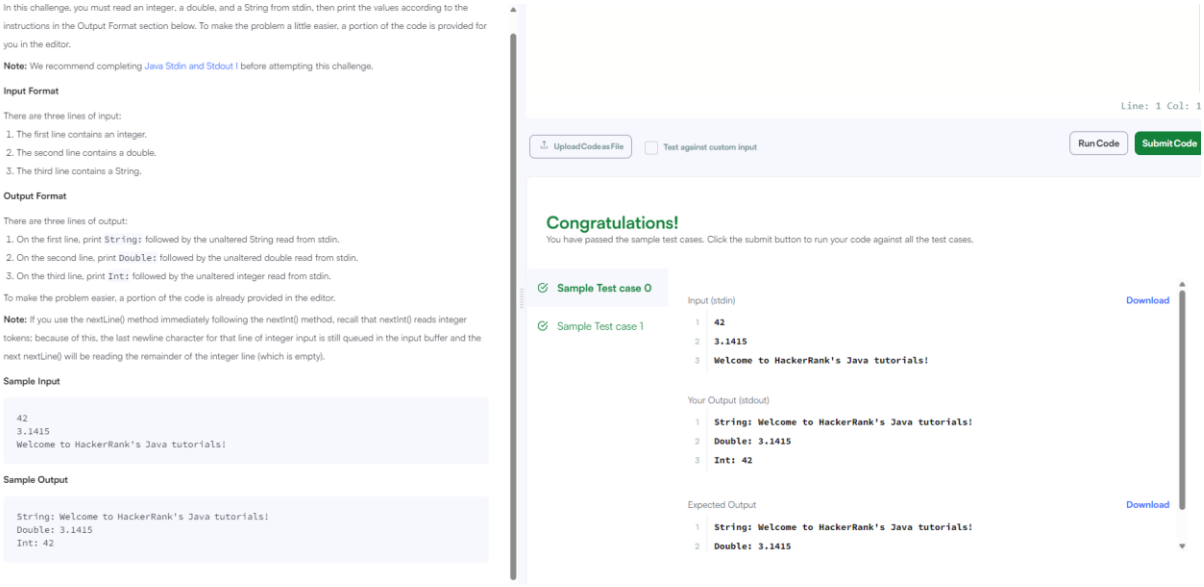
Double: 3.1415

Int: 42

CODE:

```
import java.util.Scanner;
public class Solution{
public static void main(String[] args){
Scanner sc=new Scanner(System.in);
int i=sc.nextInt();
double d=sc.nextDouble();
sc.nextLine();
String s=sc.nextLine();
System.out.println("String: "+s);
System.out.println("Double: "+d);
System.out.println("Int: "+i);
sc.close();
}
}
```

OUTPUT:



4. Java's *System.out.printf* function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using *printf*.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a *String* followed by an *integer*.
Each *String* will have a maximum of alphabetic characters, and each *integer* will be in the inclusive range from to .

Output Format

In each line of output there should be two columns:
The first column contains the *String* and is left justified using exactly characters.
The second column contains the *integer*, expressed in exactly digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Sample Input

java 100

cpp 65

python 50

Sample Output

```
================================
java         100
cpp          065
python       050
================================
```

Explanation

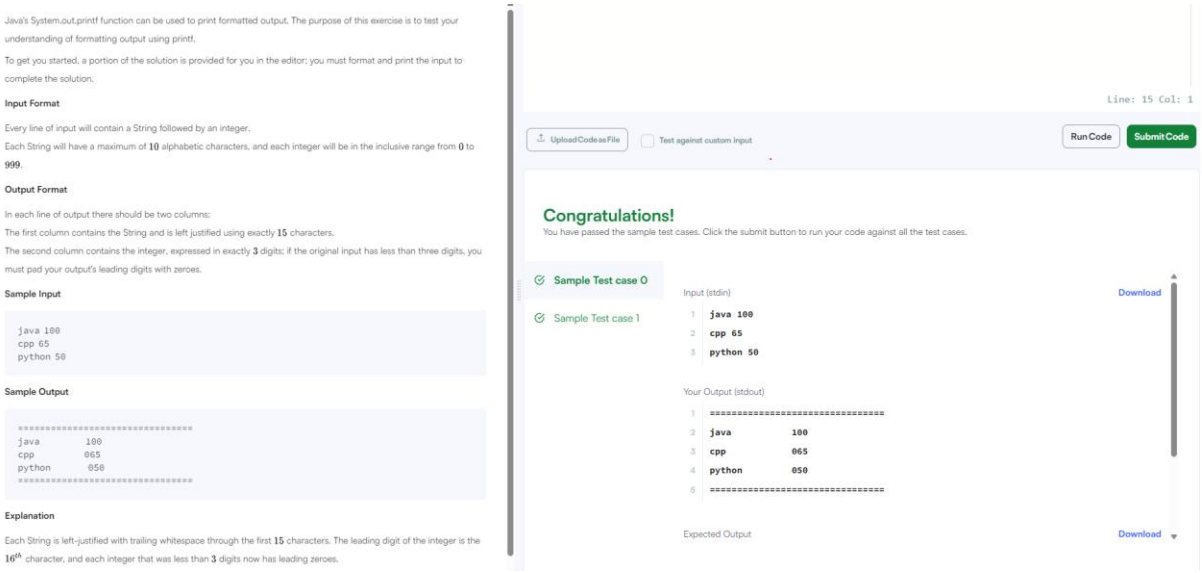
Each *String* is left-justified with trailing whitespace through the first characters. The leading digit of the *integer* is the character, and each *integer* that was less than digits now has leading zeroes.

CODE:

```
import java.util.Scanner;

public class Solution{

public static void main(String[] args){

Scanner sc=new Scanner(System.in);
```

```
System.out.println("================================");
for(int i=0;i<3;i++){
String s=sc.next();
int n=sc.nextInt();
System.out.printf("%-15s%03d%n",s,n);
}
System.out.println("================================");
sc.close();
}
}
```

OUTPUT:



5. **Objective**

In this challenge, we're going to use loops to help us do some simple math.

Task

Given an integer, , print its first multiples. Each multiple (where) should be printed on a new line in the form: N x i = result.

Input Format

A single integer, .

Constraints

-

Output Format

Print lines of output; each line (where) contains the of in the form: N x i = result.

Sample Input

2

Sample Output

2 x 1 = 2

2 x 2 = 4

2 x 3 = 6

2 x 4 = 8

2 x 5 = 10

2 x 6 = 12

2 x 7 = 14

2 x 8 = 16

2 x 9 = 18

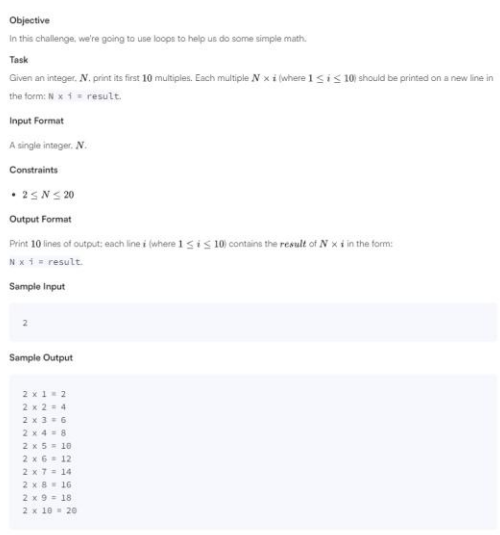
2 x 10 = 20

CODE:

```
import java.util.Scanner;

public class Solution{

public static void main(String[] args){

Scanner sc=new Scanner(System.in);

int n=sc.nextInt();

for(int i=1;i<=10;i++){
```

```
System.out.println(n+" x "+i+" = "+(n*i));
}
sc.close();
}
}
```

OUTPUT:



Objective

In this challenge, we're going to use loops to help us do some simple math.

Task

Given an integer, N, print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: `N x i = result`.

Input Format

A single integer, N.

Constraints

- $2 \leq N \leq 20$

Output Format

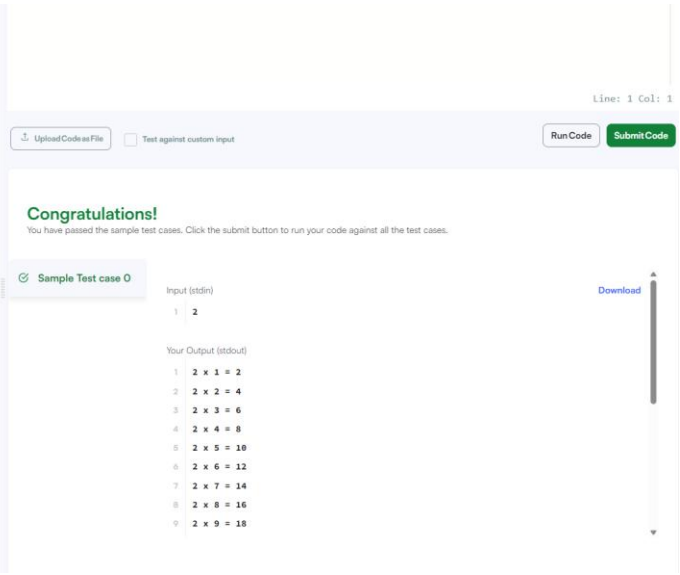
Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the *result* of $N \times i$ in the form: `N x i = result`.

Sample Input

```
2
```

Sample Output

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
```



6. We use the integers , , and to create the following series:

You are given queries in the form of , , and . For each query, print the series corresponding to the given , , and values as a single line of space-separated integers.

Input Format

The first line contains an integer, , denoting the number of queries.
Each line of the subsequent lines contains three space-separated integers describing the respective , , and values for that query.

Output Format

For each query, print the corresponding series on a new line. Each series must be printed in order as a single line of space-separated integers.

Sample Input

2

0 2 10

5 3 5

Sample Output

2 6 14 30 62 126 254 510 1022 2046

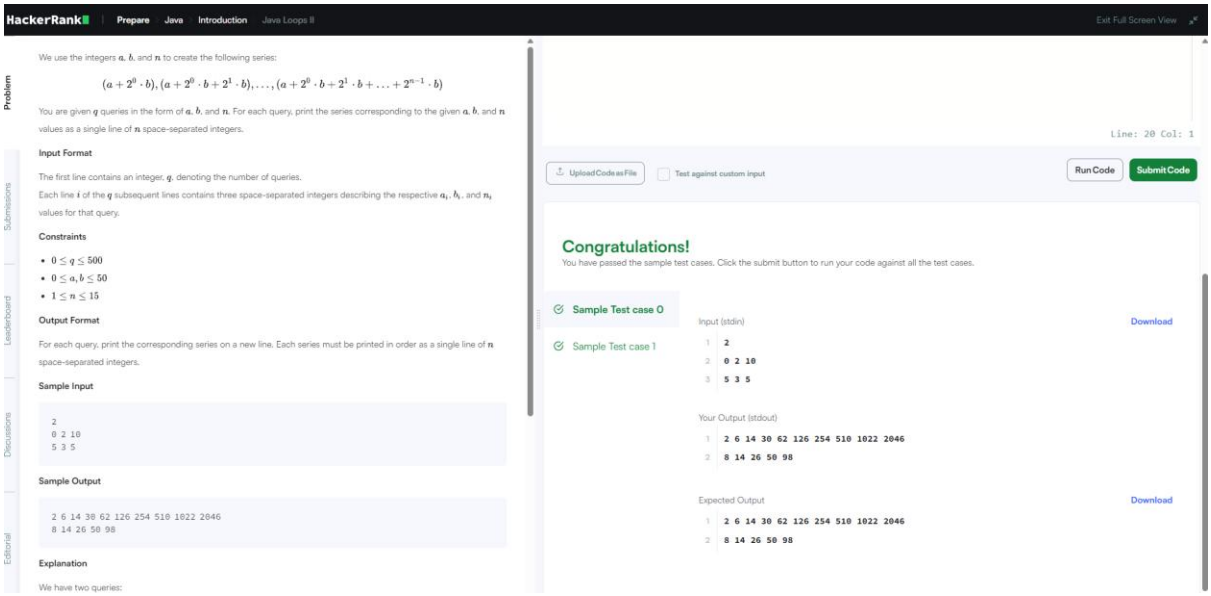
8 14 26 50 98

CODE:

```
import java.util.Scanner;

public class Solution{

public static void main(String[] args){

Scanner sc=new Scanner(System.in);

int t=sc.nextInt();

for(int i=0;i<t;i++){

int a=sc.nextInt();
```

```
int b=sc.nextInt();
int n=sc.nextInt();
int sum=a;
for(int j=0;j<n;j++){
sum=sum+(1<<j)*b;
System.out.print(sum+" ");
}
System.out.println();
}
sc.close();
}
}
```

OUTPUT:



7. Java has 8 primitive data types; *char, boolean, byte, short, int, long, float, and double*. For this exercise, we'll work with the primitives used to hold integer values (*byte, short, int,* and *long*):

- A *byte* is an 8-bit signed integer.

- A *short* is a 16-bit signed integer.
- An *int* is a 32-bit signed integer.
- A *long* is a 64-bit signed integer.

Given an input integer, you must determine which primitive data types are capable of properly storing that input.

To get you started, a portion of the solution is provided for you in the editor.

Input Format

The first line contains an integer, , denoting the number of test cases.
Each test case, , is comprised of a single line with an integer, , which can be arbitrarily large or small.

Output Format

For each input variable and appropriate primitive , you must determine if the given primitives are capable of storing it. If yes, then print:

n can be fitted in:

* dataType

If there is more than one appropriate data type, print each one on its own line and order them by size (i.e.:).

If the number cannot be stored in one of the four aforementioned primitives, print the line:

n can't be fitted anywhere.

Sample Input

5

-150

150000

1500000000

213333333333333333333333333333333333

-100000000000000

Sample Output

-150 can be fitted in:

* short

* int

* long

150000 can be fitted in:

* int

* long

1500000000 can be fitted in:

* int

* long

21333333333333333333333333333333333 can't be fitted anywhere.

-100000000000000 can be fitted in:

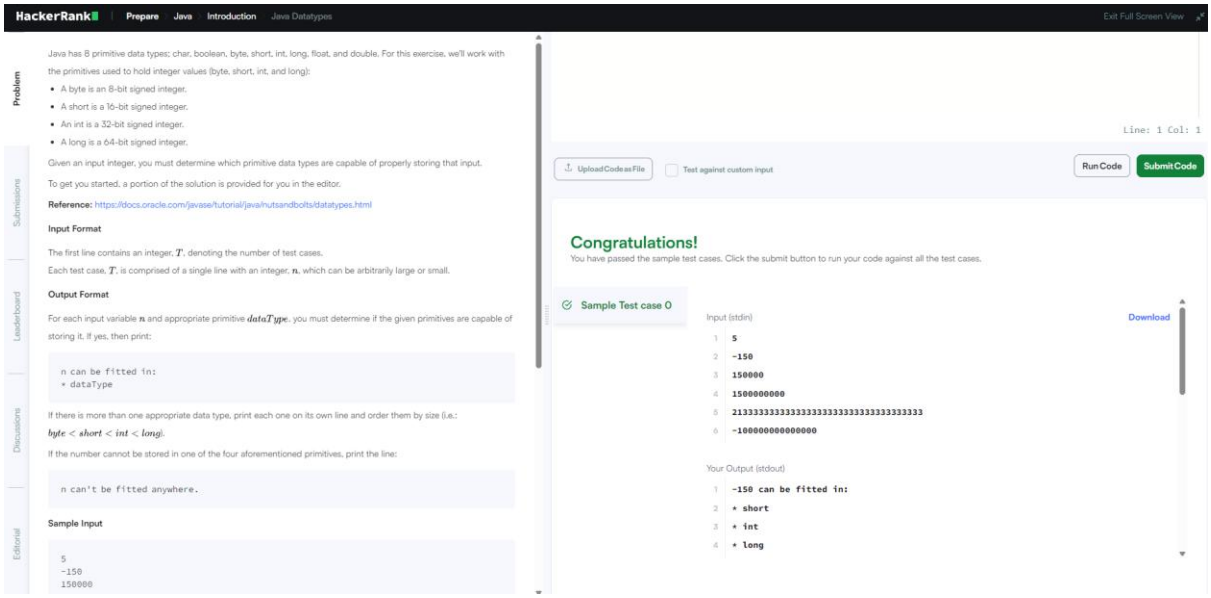
* long

CODE:

```
import java.util.Scanner;
public class Solution{
public static void main(String[] args){
Scanner sc=new Scanner(System.in);
int t=sc.nextInt();
for(int i=0;i<t;i++){
try{
```

```
long x=sc.nextLong();
System.out.println(x+" can be fitted in:");
if(x>=-128&&x<=127)
System.out.println("* byte");
if(x>=-32768&&x<=32767)
System.out.println("* short");
if(x>=-2147483648L&&x<=2147483647L)
System.out.println("* int");
System.out.println("* long");
}
catch(Exception e){
System.out.println(sc.next()+" can't be fitted anywhere.");
}
}
sc.close();
}
}
```

OUTPUT:



8. "In computing, *End Of File* (commonly abbreviated *EOF*) is a condition in a computer operating system where no more data can be read from a data source.")

The challenge here is to read lines of input until you reach *EOF*, then number and print all lines of content.

Hint: Java's *Scanner.hasNext()* method is helpful for this problem.

Input Format

Read some unknown lines of input from *stdin(System.in)* until you reach *EOF*; each line of input contains a non-empty *String*.

Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Sample Input

Hello world

I am a file

Read me until end-of-file.

Sample Output

1 Hello world

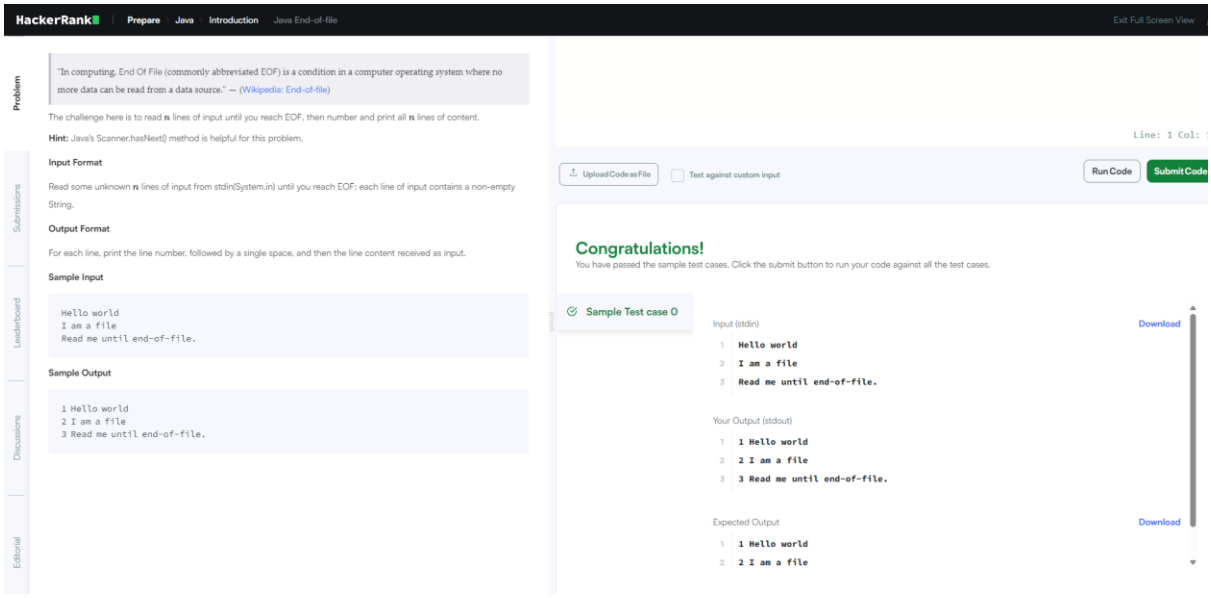
2 I am a file

3 Read me until end-of-file.

CODE:

```
import java.util.*;
public class Solution{
public static void main(String[]args){
Scanner sc=new Scanner(System.in);
int i=1;
while(sc.hasNextLine()){
System.out.println(i+" "+sc.nextLine());
i++;
}
sc.close();
}
}
```

OUTPUT:



9. Static initialization blocks are executed when the class is loaded, and you can initialize static variables in those blocks.

It's time to test your knowledge of *Static initialization blocks*. You can read about it here.

You are given a class *Solution* with a *main* method. Complete the given code so that it outputs the area of a parallelogram with breadth and height . You should read the variables from the standard input.

If or , the output should be *"java.lang.Exception: Breadth and height must be positive"* without quotes.

Input Format

There are two lines of input. The first line contains : the breadth of the parallelogram. The next line contains : the height of the parallelogram.

Output Format

If both values are greater than zero, then the *main* method must output the area of the *parallelogram*. Otherwise, print *"java.lang.Exception: Breadth and height must be positive"* without quotes.

Sample input 1

1

3

Sample output 1

3

Sample input 2

-1

2

Sample output 2

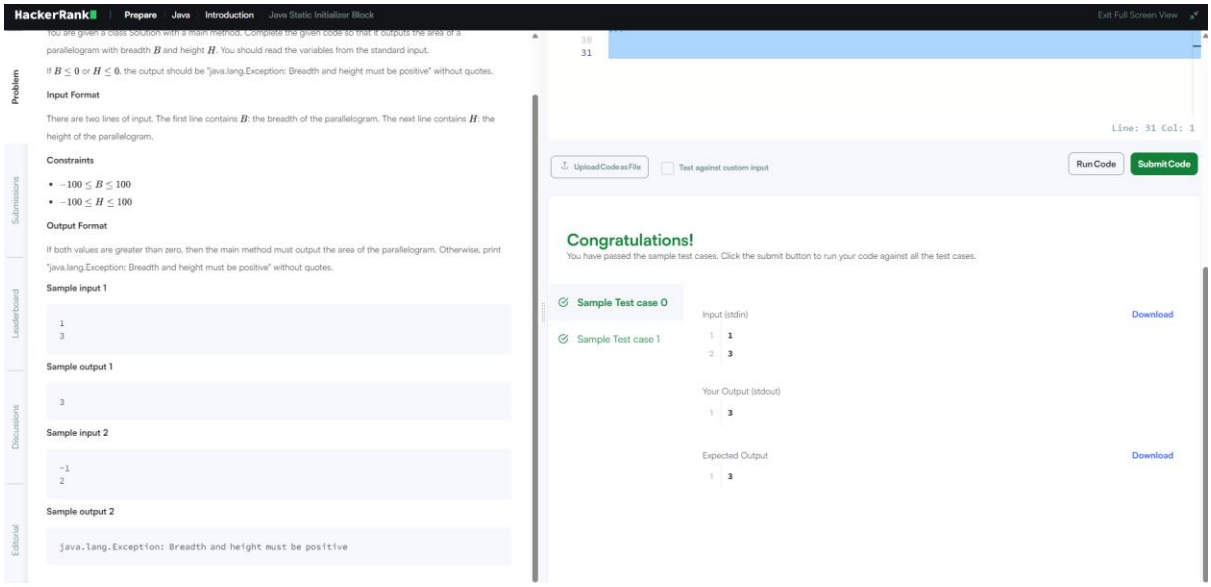
java.lang.Exception: Breadth and height must be positive

CODE:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
static int B,H;
static boolean flag=true;

static{
Scanner sc=new Scanner(System.in);
B=sc.nextInt();
H=sc.nextInt();
```

```
if(B<=0||H<=0){
flag=false;
System.out.println("java.lang.Exception: Breadth and height must be positive");
}
}

public static void main(String[] args){
      if(flag){
         int area=B*H;
         System.out.print(area);
      }

   }//end of main

}//end of class
```

OUTPUT:



10. **Given a double-precision number, , denoting an amount of money, use the NumberFormat class' getCurrencyInstance method to convert into the US, Indian, Chinese, and French currency formats. Then print the formatted values as follows:**

US: formattedPayment

India: formattedPayment

China: formattedPayment

France: formattedPayment

where is formatted according to the appropriate Locale's currency.

Note: India does not have a built-in Locale, so you must construct one where the language is en (i.e., English).

Input Format

A single double-precision number denoting .

Constraints

-

Output Format

On the first line, print US: u where is formatted for US currency.
On the second line, print India: i where is formatted for Indian currency.
On the third line, print China: c where is formatted for Chinese currency.
On the fourth line, print France: f, where is formatted for French currency.

Sample Input

12324.134

Sample Output

US: $12,324.13

India: Rs.12,324.13

China: ￥12,324.13

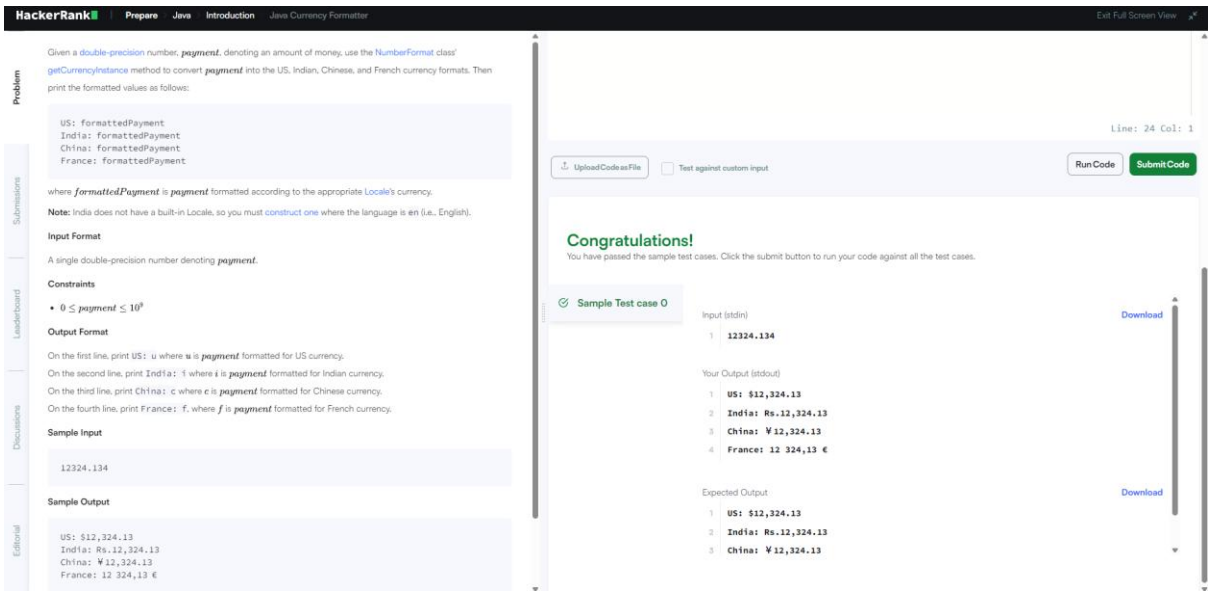
France: 12 324,13 €

CODE:

```
import java.util.*;
import java.text.*;

public class Solution {

 public static void main(String[] args) {
 Scanner scanner = new Scanner(System.in);
  double payment = scanner.nextDouble();
  scanner.close();
  NumberFormat us = NumberFormat.getCurrencyInstance(Locale.US);
      NumberFormat china =
NumberFormat.getCurrencyInstance(Locale.CHINA);
```

```
        NumberFormat france =
NumberFormat.getCurrencyInstance(Locale.FRANCE);


                    // Create Indian Locale

                        Locale indiaLocale = new Locale("en", "IN");

 NumberFormat india = NumberFormat.getCurrencyInstance(indiaLocale);


    System.out.println("US: " + us.format(payment));

        System.out.println("India: " + india.format(payment));

   System.out.println("China: " + china.format(payment));

   System.out.println("France: " + france.format(payment));

                                                    }

                                                    }
```

OUTPUT:



11. Write the following code in your editor below:

1. A class named *Arithmetic* with a method named *add* that takes integers as parameters and returns an integer denoting their sum.
2. A class named *Adder* that inherits from a superclass named *Arithmetic*.

Your classes should not be be .

Input Format

You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the *add* method on an *Adder* object and passing it integer parameters.

Output Format

You are not responsible for printing anything to stdout. Your *add* method must return the sum of its parameters.

Sample Output

The *main* method in the *Solution* class above should print the following:

My superclass is: Arithmetic

42 13 20

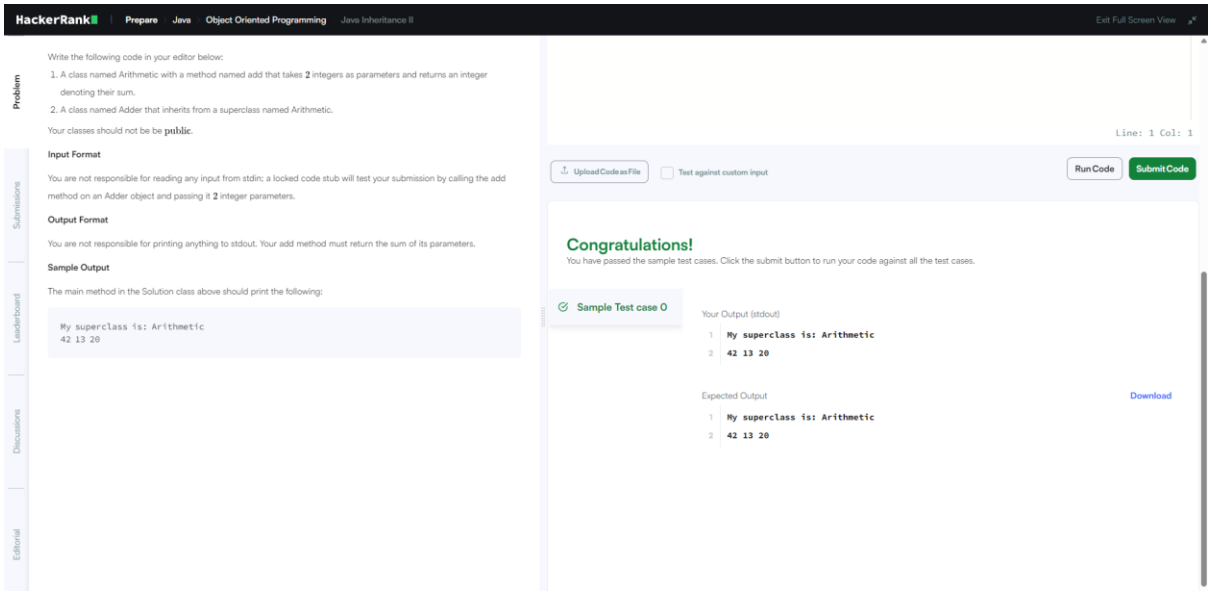
CODE:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;
```

```
import java.util.regex.*;


class Arithmetic {
      public int add(int a, int b) {
            return a + b;
              }
              }


              class Adder extends Arithmetic {


              }

public class Solution{
   public static void main(String []args){
      // Create a new Adder object
      Adder a = new Adder();


      // Print the name of the superclass on a new line
      System.out.println("My superclass is: " +
a.getClass().getSuperclass().getName());


      // Print the result of 3 calls to Adder's `add(int,int)` method as 3 space-
separated integers:
      System.out.print(a.add(10,32) + " " + a.add(10,3) + " " + a.add(10,10) +
"\n");
    }
```

}

OUTPUT:


HackerRank
Prepare
Java
Object Oriented Programming
Java Inheritance II
Exit Full Screen View
Problem
Write the following code in your editor below:
1. A class named Arithmetic with a method named add that takes 2 integers as parameters and returns an integer denoting their sum.
2. A class named Adder that inherits from a superclass named Arithmetic.
Your classes should not be be public.
Input Format
You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the add method on an Adder object and passing it 2 integer parameters.
Output Format
You are not responsible for printing anything to stdout. Your add method must return the sum of its parameters.
Sample Output
The main method in the Solution class above should print the following:
My superclass is: Arithmetic
42 13 20
Submissions
Leaderboard
Discussions
Editorial
Line: 1 Col: 1
Upload Code as File
Test against custom input
Run Code
Submit Code
Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.
Sample Test case 0
Your Output (stdout)
My superclass is: Arithmetic
42 13 20
Expected Output
Download
My superclass is: Arithmetic
42 13 20

12. A Java abstract class is a class that can't be instantiated. That means you cannot create new instances of an abstract class. It works as a base for subclasses. You should learn about Java Inheritance before attempting this challenge.

Following is an example of abstract class:

```
abstract class Book{
   String title;
   abstract void setTitle(String s);
   String getTitle(){
      return title;
   }
}
```

If you try to create an instance of this class like the following line you will get an error:

```
Book new_novel=new Book();
```

You have to create another class that extends the abstract class. Then you can create an instance of the new class.

Notice that *setTitle* method is abstract too and has no body. That means you must implement the body of that method in the child class.

In the editor, we have provided the abstract *Book* class and a *Main* class. In the Main class, we created an instance of a class called *MyBook*. Your task is to write just the *MyBook* class.

Your class mustn't be public.

Sample Input

A tale of two cities

Sample Output

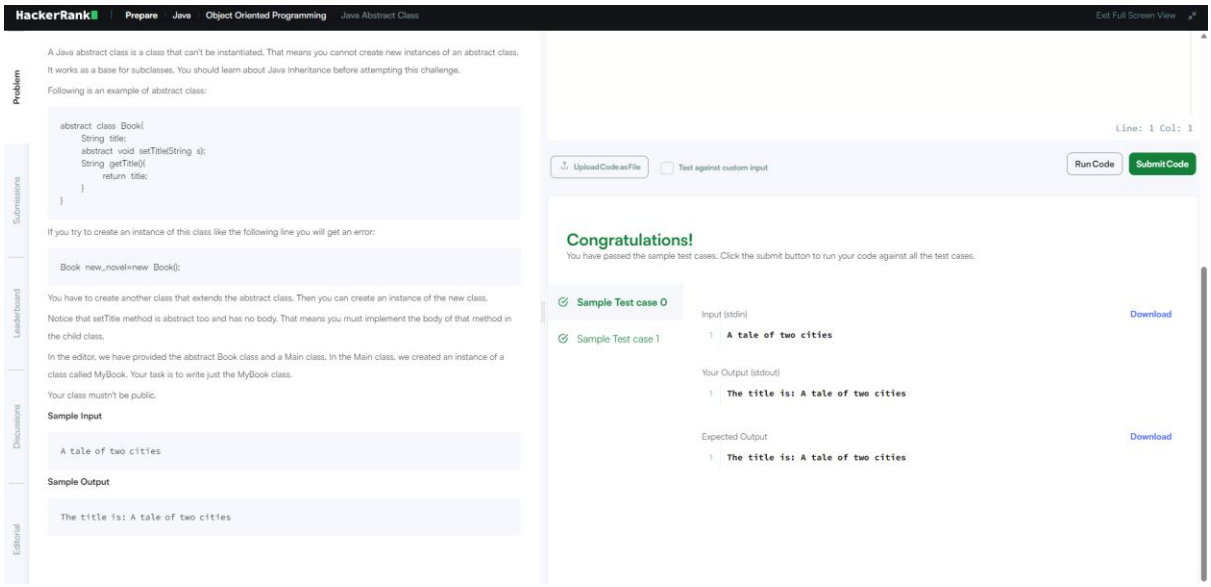
The title is: A tale of two cities

CODE:

```
import java.util.*;
abstract class Book{
   String title;
   abstract void setTitle(String s);
   String getTitle(){
      return title;
   }
}

class MyBook extends Book {
      void setTitle(String s) {
            title = s;
               }
               }


public class Main{

   public static void main(String []args){
```

```
        //Book new_novel=new Book(); This line prHMain.java:25: error: Book is abstract; cannot be instantiated
        Scanner sc=new Scanner(System.in);
        String title=sc.nextLine();
        MyBook new_novel=new MyBook();
        new_novel.setTitle(title);
        System.out.println("The title is: "+new_novel.getTitle());
        sc.close();


    }
}
```

OUTPUT:



Q13. Java Currency Formatter

Java Code

```
import java.text.NumberFormat;
import java.util.Locale;
```

```
public class Solution {

   public static void main(String[] args) {

      double payment = 12324.134;

      NumberFormat us = NumberFormat.getCurrencyInstance(Locale.US);
      NumberFormat india = NumberFormat.getCurrencyInstance(new
Locale("en", "IN"));
      NumberFormat china =
NumberFormat.getCurrencyInstance(Locale.CHINA);
      NumberFormat france =
NumberFormat.getCurrencyInstance(Locale.FRANCE);

      System.out.println("US: " + us.format(payment));
      System.out.println("India: " + india.format(payment));
      System.out.println("China: " + china.format(payment));
      System.out.println("France: " + france.format(payment));
   }
}
```

Output

US: $12,324.13

India: ₹12,324.13

China: ¥12,324.13

France: 12 324,13 €

Q14. Java Strings Introduction

Java Code

```
import java.util.*;

public class Solution {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      String A = sc.next();
      String B = sc.next();

      System.out.println(A.length() + B.length());

      if (A.compareTo(B) > 0)
         System.out.println("Yes");
      else
         System.out.println("No");

      System.out.println(
            A.substring(0,1).toUpperCase() + A.substring(1)
            + " " +
            B.substring(0,1).toUpperCase() + B.substring(1));
```

```
        sc.close();
    }
}
```

Sample Output

```
10
No
Hello Java
```

Q15. Java Substring

Java Code

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in)
        String s = sc.next();
        int start = sc.nextInt();
        int end = sc.nextInt();
        System.out.println(s.substring(start, end));
        sc.close();
    }
}
```

Sample Output

```
wel
```

Q16. Java Substring Comparisons

Java Code

```
public class Solution {
   public static void main(String[] args)
      String s = "welcometojava";
      int k = 3;

      String smallest = s.substring(0, k);
      String largest = s.substring(0, k);
      for (int i = 1; i <= s.length() - k; i++) {
         String sub = s.substring(i, i + k);
         if (sub.compareTo(smallest) < 0)
            smallest = sub;
         if (sub.compareTo(largest) > 0)
            largest = sub;
      }
      System.out.println(smallest);
      System.out.println(largest);
   }
}
```

Output

```
ava
wel
```

Q17. Java String Reverse

Java Code

```
import java.util.*;
public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      String s = sc.next();
      String rev = new StringBuilder(s).reverse().toString();
      if (s.equals(rev))
         System.out.println("Yes");
      else
         System.out.println("No");
      sc.close();
   }
}
```

Sample Output

Yes

Q18. Java Anagrams

Java Code

```
import java.util.*;
public class Solution {
   public static void main(String[] args)
      Scanner sc = new Scanner(System.in);
      char[] a = sc.next().toLowerCase().toCharArray();
      char[] b = sc.next().toLowerCase().toCharArray();
      Arrays.sort(a);
```

```
        Arrays.sort(b)
        if (Arrays.equals(a, b))
            System.out.println("Anagrams");
        else
            System.out.println("Not Anagrams");
        sc.close();
    }
}
```

Sample Output

Anagrams

Q19. Java String Tokens

Java Code

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().trim();
        if (s.isEmpty()) {
            System.out.println(0);
        } else {
            String[] arr = s.split("[ !,?._'@]+");
            System.out.println(arr.length);
            for (String str : arr)
                System.out.println(str);
```

```
        }
        sc.close();
    }
}
```

Sample Output

```
5
He
is
a
good
boy
```

Q20. Pattern Syntax Checker

Java Code

```
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        String[] patterns = {
            "[AZ[a-z](a-z)",
            "batcatpat",
            "[a-zA-Z]+"
        };
        for (String pattern : patterns) {
            try {
                Pattern.compile(pattern);
```

```
            System.out.println("Valid");
        } catch (PatternSyntaxException e) {
            System.out.println("Invalid");
        }
    }
  }
}
```

Sample Output

```
Invalid
Valid
Valid
```

Q21. Valid Username Regular Expression

Java Code

```
import java.util.regex.*;
public class Solution {
   public static void main(String[] args) {
      String[] usernames = {
         "Julia",
         "Samantha_21",
         "1Samantha",
         "Samantha?10_2A",
         "JuliaZ007"
      };
```

```
        String regex = "^[A-Za-z][A-Za-z0-9_]{7,29}$";
        Pattern pattern = Pattern.compile(regex);
        for (String username : usernames) {
            if (pattern.matcher(username).matches()) {
                System.out.println("Valid");
            } else {
                System.out.println("Invalid");
            }
        }
    }
}
```

Sample Output

```
Invalid
Valid
Invalid
Invalid
Valid
```

Q22. Tag Content Extractor

Java Code

```
import java.util.regex.*;
public class Solution {
    public static void main(String[] args) {
        String[] input = {
            "<h1>Nayeem loves counseling</h1>",
```

```
        "<h1><h1>Sanjay has no watch</h1></h1><par>So wait for a
while</par>",
        "<Amee>safat codes like a ninja</amee>",
        "<SA premium>Imtiaz has a secret crush</SA premium>"
    };
    String regex = "<(.+)>([^<]+)</\\1>";
    Pattern pattern = Pattern.compile(regex);
    for (String line : input) {
      Matcher matcher = pattern.matcher(line);
      boolean found = false;
      while (matcher.find()) {
        System.out.println(matcher.group(2));
        found = true;
      }
      if (!found) {
        System.out.println("None");
      }
    }
  }
}
```

Sample Output

```
Nayeem loves counseling
Sanjay has no watch
So wait for a while
None
```

None

Q23. Java BigDecimal

Java Code

```
import java.math.BigDecimal;
import java.util.Arrays;
import java.util.Comparator;
public class Solution {
   public static void main(String[] args) {
      String[] numbers = {
         "-100",
         "50",
         "0",
         "56.6",
         "90",
         "0.12",
         ".12",
         "02.34",
         "000.000"
      };

      Arrays.sort(numbers, new Comparator<String>() {
         @Override
         public int compare(String a, String b) {
            BigDecimal bd1 = new BigDecimal(a);
```

```
            BigDecimal bd2 = new BigDecimal(b);
            return bd2.compareTo(bd1);
        }
    });
    for (String num : numbers) {
        System.out.println(num);
    }
  }
}
```

Sample Output

```
90
56.6
50
02.34
0.12
.12
000.000
0
-100
```

Q24. Java Primality Test

Java Code

```
import java.math.BigInteger;
import java.util.Scanner;
public class Solution {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BigInteger n = sc.nextBigInteger();
        if (n.isProbablePrime(1)) {
            System.out.println("prime");
        } else {
            System.out.println("not prime");
        }
        sc.close();
    }
}
```

Sample Input

13

Sample Output

prime

Another Sample Input

20

Sample Output

not prime

Q25. Java BigInteger

Java Code

```
import java.math.BigInteger;
```

```
import java.util.Scanner;
public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      BigInteger a = sc.nextBigInteger();
      BigInteger b = sc.nextBigInteger();
      System.out.println(a.add(b));
      System.out.println(a.multiply(b));
      sc.close();
   }
}
```

Sample Input

1234

20

Sample Output

1254

24680

Another Sample Input

100000000000000000000000000000

100000000000000000000000000000

Sample Output

200000000000000000000000000000

10000000000000000000000000000000000000000000000000000000000000

Q26. Java 1D Array

Java Code

```
import java.util.Scanner;
public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      int[] arr = new int[n];
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }
      for (int i = 0; i < n; i++) {
         System.out.println(arr[i]);
      }
      sc.close();
   }
}
```

Sample Input

```
5
10
20
30
40
50
```

Sample Output

10

20

30

40

50

Q27. Java 2D Array

Java Code

```
import java.util.Scanner;
public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int[][] arr = new int[6][6];
      for (int i = 0; i < 6; i++) {
         for (int j = 0; j < 6; j++) {
            arr[i][j] = sc.nextInt();
         }
      }

      int maxSum = Integer.MIN_VALUE;
      for (int i = 0; i <= 3; i++) {
         for (int j = 0; j <= 3; j++) {
            int sum = arr[i][j] + arr[i][j + 1] + arr[i][j + 2]
                  + arr[i + 1][j + 1]
```

```
                + arr[i + 2][j] + arr[i + 2][j + 1] + arr[i + 2][j + 2];
            if (sum > maxSum) {
                maxSum = sum;
            }
        }
    }

    System.out.println(maxSum);
    sc.close();
  }
}
```

Sample Input

```
1 1 1 0 0 0
0 1 0 0 0 0
1 1 1 0 0 0
0 0 2 4 4 0
0 0 0 2 0 0
0 0 1 2 4 0
```

Sample Output

```
19
```

Q28. Java Subarray

Java Code

```
import java.util.Scanner;
public class Solution {
```

```
    public static void main(String[] args)
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int count = 0
        for (int i = 0; i < n; i++)
            int sum = 0
            for (int j = i; j < n; j++) {
                sum += arr[j
                if (sum < 0) {
                    count++;
                }
            }
        }

        System.out.println(count);
        sc.close();
    }
}
```

Sample Input

5

1 -2 4 -5 1

Sample Output

9

Q29. Java ArrayList

Java Code

```
import java.util.ArrayList;
import java.util.Scanner;
public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();
      ArrayList<ArrayList<Integer>> list = new ArrayList<>();
      for (int i = 0; i < n; i++) {
         int d = sc.nextInt();
         ArrayList<Integer> row = new ArrayList<>();
         for (int j = 0; j < d; j++) {
            row.add(sc.nextInt());

         list.add(row);
      }
      int q = sc.nextInt();
      while (q-- > 0)
         int x = sc.nextInt();
         int y = sc.nextInt();
         try {
```

```
            System.out.println(list.get(x - 1).get(y - 1));
        } catch (Exception e) {
            System.out.println("ERROR!");
        }
      }
      sc.close();
   }
}
```

Sample Input

```
5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
5
1 3
3 4
3 1
4 3
5 5
```

Sample Output

```
74
52
37
```

ERROR!

ERROR!

Q30. Java List

Java Code

```
import java.util.ArrayList;
import java.util.Scanner;
public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      ArrayList<Integer> list = new ArrayList<>();
      for (int i = 0; i < n; i++) {
         list.add(sc.nextInt());
      }
      int q = sc.nextInt();
      while (q-- > 0) {
         String operation = sc.next();
         if (operation.equals("Insert")) {
            int index = sc.nextInt();
            int value = sc.nextInt();
            list.add(index, value);
         } else if (operation.equals("Delete"))
            int index = sc.nextInt();
            list.remove(index);
         }
```

```
        }
        for (int num : list) {
            System.out.print(num + " ");
        }
        sc.close();
    }
}
```

Sample Input

5

12 0 1 78 12

2

Insert

5 23

Delete

0

Sample Output

0 1 78 12 23